



20 October 2025 | M2 SAR | Paris, France

---

# LTL Model Checking

M2 SAR - UE Applications et systèmes temps réel répartis embarqués

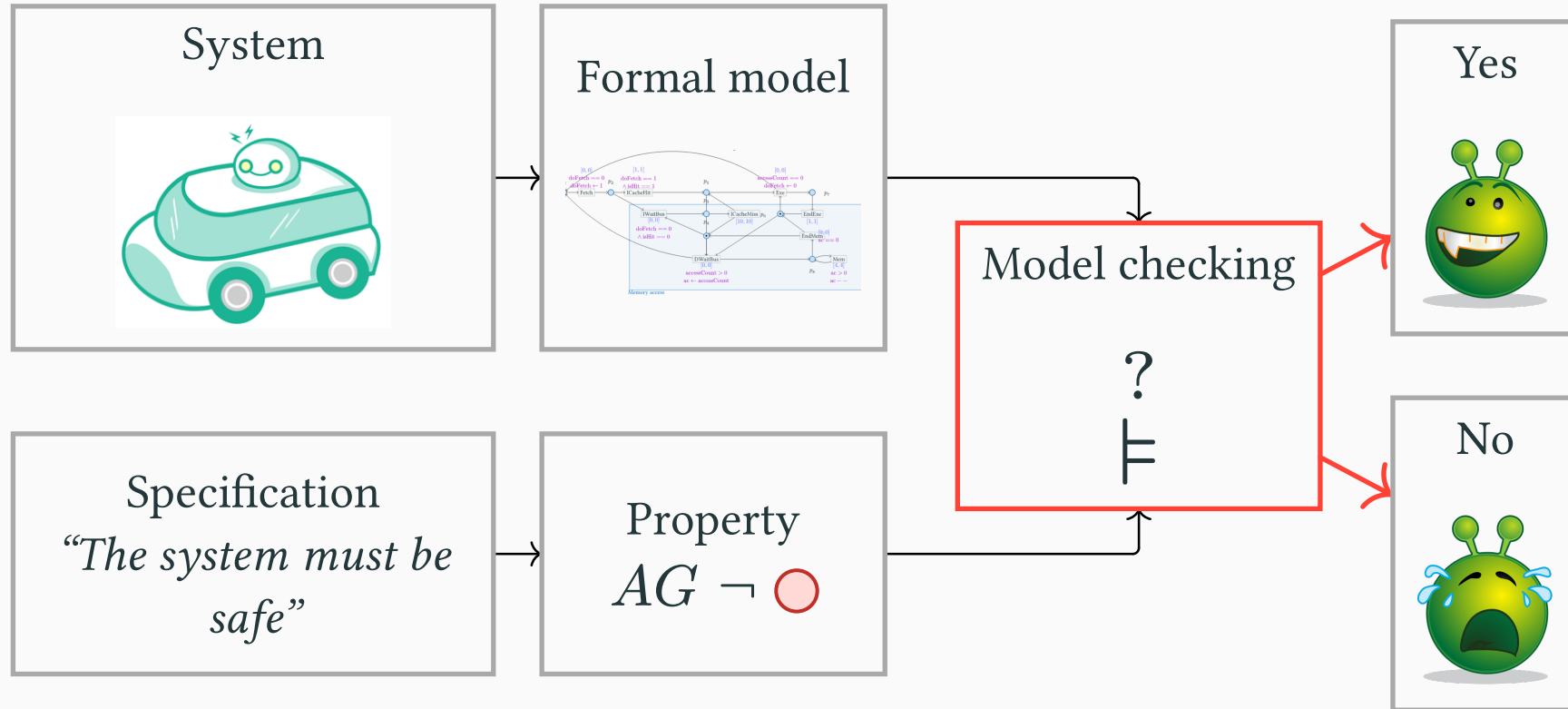
---

Dylan Marinho

Sorbonne Université, CNRS UMR 7606, LIP6



# Modelisation



**LTL**

Büchi automata

Constructing  $\mathcal{B}_{\neg\varphi}$

Validating the formula



Fairness

LTL

CTL

**LTL**

---

## Checking whether $\mathcal{A} \models \varphi$

1. Transform  $\varphi$  in an automaton  $\mathcal{B}_\varphi$  s.t.  $\mathcal{L}(\mathcal{B}_\varphi) = [\varphi]$
2. Test if  $\mathcal{L}(\mathcal{A}) \subseteq \overline{\mathcal{L}(\mathcal{B}_\varphi)}$ , i.e.  $\mathcal{L}(\mathcal{A}) \cap \overline{\mathcal{L}(\mathcal{B}_\varphi)} \neq \emptyset$ 
  - if empty, then  $\mathcal{A} \models \varphi$

LTL

Büchi automata

Constructing  $\mathcal{B}_{-\varphi}$

Validating the formula



Fairness

LTL

CTL

# Büchi automata

---

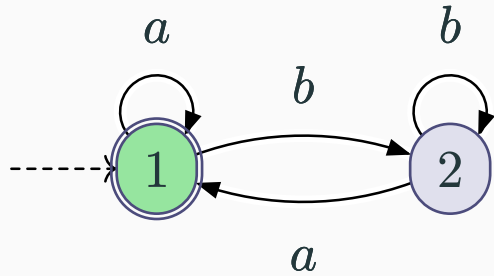
## Definition 1 (Büchi automaton)

A **Büchi automaton** is a quintuplet  $(Q, \Sigma, I, F, T)$  with

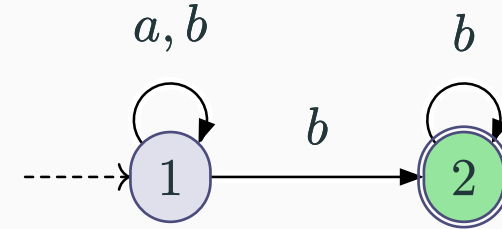
- ▶ a set of states  $Q$
- ▶ an alphabet of actions  $\Sigma$
- ▶ a *set of initial states*  $I \subseteq Q$
- ▶ a set of final states  $F \subseteq Q$
- ▶ a transition relation  $T \subseteq Q \times \Sigma \times Q$



- ▶ An **execution** of  $\mathcal{A}$  on an *infinite* word  $w = w_0w_1w_2\dots$  over  $\Sigma$  is a sequence  $r = q_0q_1q_2q_3\dots$  such that  $q_0 \in I$  and  $(q_i, w_i, q_{i+1}) \in T$  for all  $i$
- ▶  $r$  is **accepting** if  $q_i \in F$  for **an infinite number** of  $i$
- ▶  $w$  is **accepted** by  $\mathcal{A}$  if there exists an accepting execution of  $\mathcal{A}$  on  $w$
- ▶  $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^\omega \mid w \text{ is accepted by } \mathcal{A}\}$



$$\mathcal{L}(\mathcal{A}) = \{w \in \{a, b\}^\omega \mid |w|_a = \omega\}$$



$$\mathcal{L}(\mathcal{A}) = \{w \in \{a, b\}^\omega \mid w = w' \cdot bbbb\ldots\}$$

# Non-deterministic Büchi automaton

- ▶ Non-deterministic Büchi automata are **more expressive** than deterministic ones
- ▶ Every LTL formula  $\varphi$  can be transformed into a **non-deterministic** Büchi automaton  $\mathcal{B}_\varphi$  recognizing all executions satisfying  $\varphi$   
 $\mathcal{L}(\mathcal{B}_\varphi) = [\varphi]$

Non-deterministic Büchi automata are more expressive than LTL: it is not possible to transform every non-deterministic Büchi automaton into an equivalent LTL formula

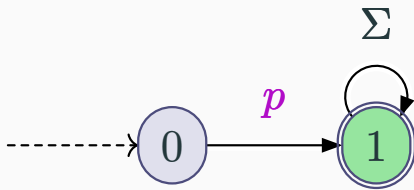
*e.g. “the event occurs one time out of two”*



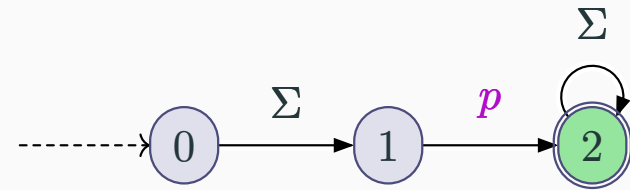


Construct Büchi automata recognizing:

$p$



$\neg p$





Construct Büchi automata recognizing:

- ▶  $Fp$
- ▶  $XXp$
- ▶  $Gp$
- ▶  $FGp$
- ▶  $GFp$
- ▶  $pUq$
- ▶  $pRq$

# Operations on Büchi automata

## Theorem 2

Büchi automata are closed under union, intersection and complementation



## Theorem 3

The emptiness problem for Büchi automata is decidable



However, the complementation operation is (very) complex

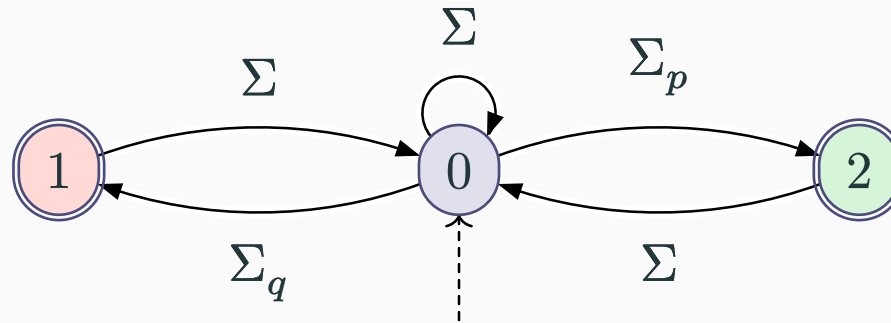
## Definition 4 (Generalized Büchi automaton)

A **generalized** Büchi automaton is a quintuplet  $(Q, \Sigma, I, F, T)$  with

- ▶ a set of states  $Q$
- ▶ an alphabet of actions  $\Sigma$
- ▶ a set of initial states  $I \subseteq Q$
- ▶ a transition relation  $T \subseteq Q \times \Sigma \times Q$
- ▶ a **set of sets** of final states  $F = \{F_1, \dots, F_k\} \subseteq 2^Q$



- ▶ A sequence  $r = q_0 q_1 q_2 q_3 \dots$  is **accepting** if for each  $\mathcal{F} \in I$ ,  $q_i \in \mathcal{F}$  for **an infinite number** of  $i$



$$\text{GF}_p \wedge \text{GF}_q$$

$$F = \{\{1\}, \{2\}\}$$

## Definition 5 (Transition-Based Generalized Büchi automaton)

A **transition-based** generalized Büchi automaton is a quintuplet  $(Q, \Sigma, I, F, T)$  with

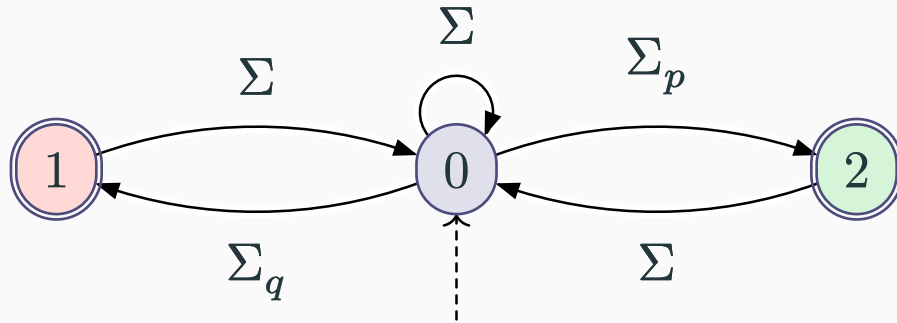
- ▶ a set of states  $Q$
- ▶ an alphabet of actions  $\Sigma$
- ▶ a set of initial states  $I \subseteq Q$
- ▶ a transition relation  $T \subseteq Q \times \Sigma \times Q$
- ▶ a **set of set of accepting transitions**  $F = \{F_1, \dots, F_k\} \subseteq 2^T$





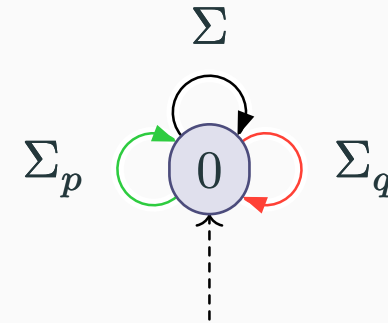
$$GFp \wedge GFq$$

Generalized BA



$$F = \{\{1\}, \{2\}\}$$

Transition-Based GBA



$$F = \{\{(0, \Sigma_p, 0)\}, \{(0, \Sigma_q, 0)\}\}$$

## Checking whether $\mathcal{A} \models \varphi$

1. Construct the automaton  $\mathcal{B}_{\neg\varphi}$  recognizing all executions **not satisfying**  $\varphi$   
 $\mathcal{L}(\mathcal{B}_{\neg\varphi}) = [\neg\varphi]$
2. Test if  $\mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{B}_{\neg\varphi}) = \emptyset$ 
  - ▶ Construct the product automaton  $\mathcal{A} \times \mathcal{B}_{\neg\varphi}$
  - ▶ Check if  $\mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{B}_{\neg\varphi}) = \mathcal{L}(\mathcal{A} \times \mathcal{B}_{\neg\varphi})$  is empty
    - ▶ if empty, then  $\mathcal{A} \models \varphi$



LTL

Büchi automata

**Constructing**  $\mathcal{B}_{\neg\varphi}$

Validating the formula



Fairness

LTL

CTL

# Constructing $\mathcal{B}_{\neg\varphi}$

---

**LTL**

Büchi automata

**Constructing**  $\mathcal{B}_{\neg\varphi}$

Validating the formula



Fairness

LTL

CTL

# Formula reduction



# Negative normal form

## Definition 1 (Negative normal form of LTL formula)

$\varphi ::= \top \mid \perp \mid p \mid \neg p \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid X\varphi \mid \varphi U \varphi \mid \varphi R \varphi$



## Definition 2 (Semantics of LTL)

|                             |                               |
|-----------------------------|-------------------------------|
| $\neg\neg p$                | $p$                           |
| $\neg(\varphi \vee \psi)$   | $\neg\varphi \wedge \neg\psi$ |
| $\neg(\varphi \wedge \psi)$ | $\neg\varphi \vee \neg\psi$   |

|                        |                          |
|------------------------|--------------------------|
| $\neg(X\varphi)$       | $X\neg\varphi$           |
| $\neg(\varphi U \psi)$ | $\neg\varphi R \neg\psi$ |
| $\neg(\varphi R \psi)$ | $\neg\varphi U \neg\psi$ |





Transform  $\mathbf{G}(p \implies \mathbf{F}q)$  in negative normal form.

# Reducing temporal operators

- ▶ We construct a graph where nodes represent sets of formulae respected by the next step of the sequence, and sets of sub-formulae that it *promises* to satisfy at the next state

→ We want atomic propositions (or their negations), and sub-formulae starting with **X**

## Definition 3 (Reduced formula set)

A set of LTL formulae  $Z$  in negative normal form is **reduced** if for all  $\varphi \in Z$ ,

$$\varphi ::= p \mid \neg p \mid X\psi$$



## Definition 4 (Consistent formula set)

A reduced set of LTL formulae  $Z$  in negative normal form is **consistent** if

- ▶  $\perp \notin Z$
- ▶  $\{p, \neg p\} \not\subseteq Z$  for all  $p \in AP$ .



# Reducing temporal operators

To reduce a formula set  $Z$  in negative normal form, we can use the following rules:

- ▶  $\varphi \mathbf{U} \psi \equiv \psi \vee (\varphi \wedge \mathbf{X}(\varphi \mathbf{U} \psi))$
  - ▶  $\varphi \mathbf{R} \psi \equiv (\varphi \wedge \psi) \vee (\psi \wedge \mathbf{X}(\varphi \mathbf{R} \psi))$
- ▶ We construct a reduction graph, where nodes are sets of formulae in negative normal form, and edges are the reduction rules.

LTL

Büchi automata

**Constructing**  $\mathcal{B}_{\neg\varphi}$

Validating the formula



Fairness

LTL

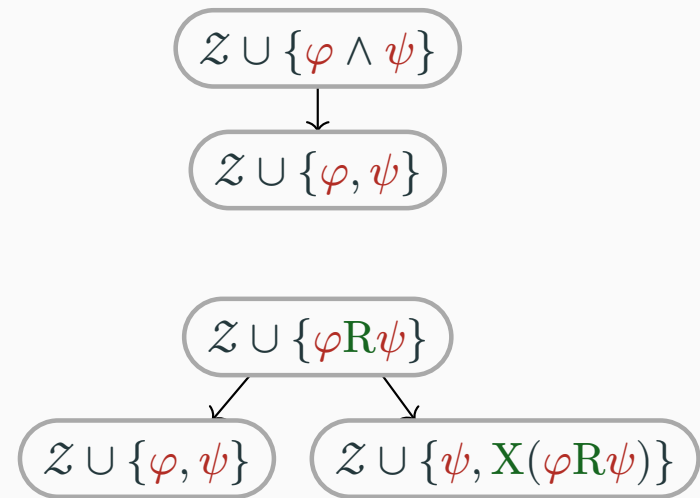
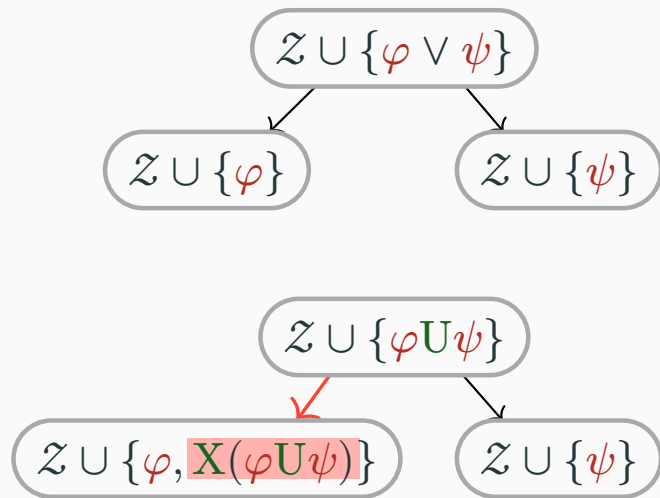
CTL

# Constructing a reduction graph





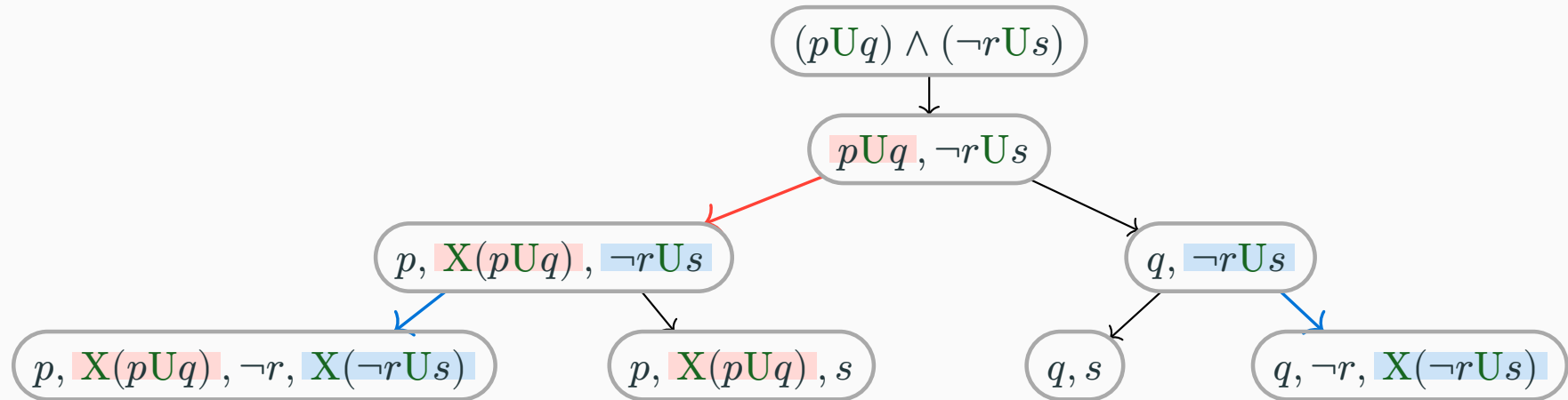
# Constructing a reduction graph



# Example: Reducing temporal operators



We reduce the *negative-normal-form* formula  $(p\mathbf{U}q) \wedge (\neg r\mathbf{U}s)$



LTL

Büchi automata

**Constructing**  $\mathcal{B}_{\neg\varphi}$

Validating the formula

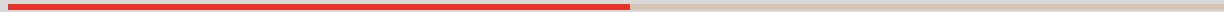


Fairness

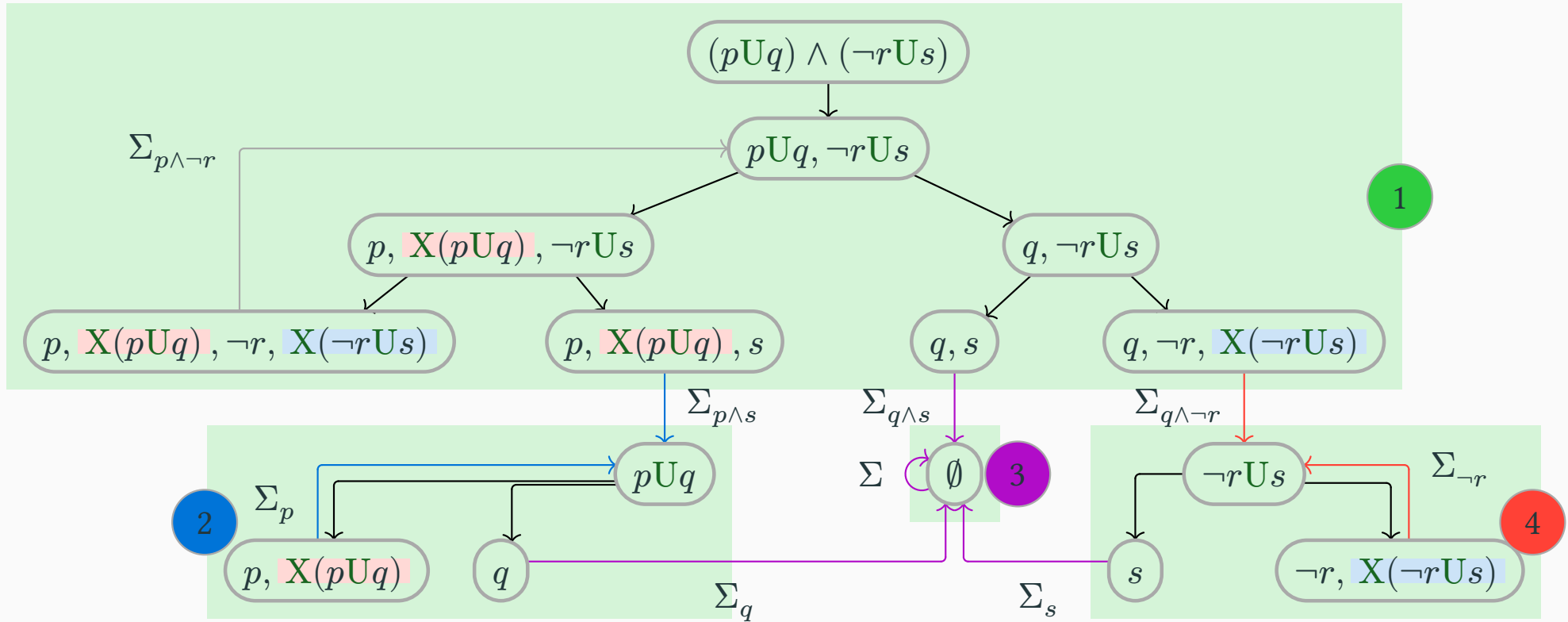
LTL

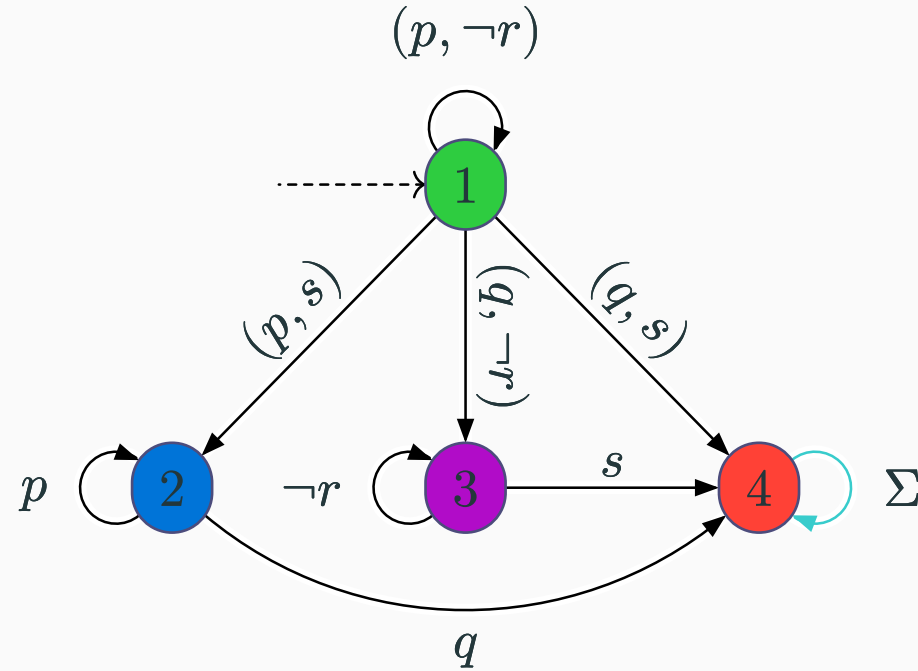
CTL

# From the reduction graph to the automaton



# Example: Transforming a formula into an automaton





$$\begin{cases} \mathcal{T}_{p \mathbf{U} q} = \{(1, 3), (3, 3), (3, 4), (1, 4), (2, 4), (4, 4)\} \\ \mathcal{T}_{\neg r \mathbf{U} s} = \{(1, 2), (2, 2), (2, 4), (1, 4), (4, 4)\} \end{cases}$$

**LTL**

Büchi automata

**Constructing**  $\mathcal{B}_{\neg\varphi}$

Validating the formula

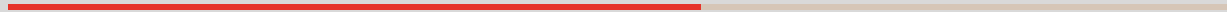


Fairness

LTL

CTL

# Exercise

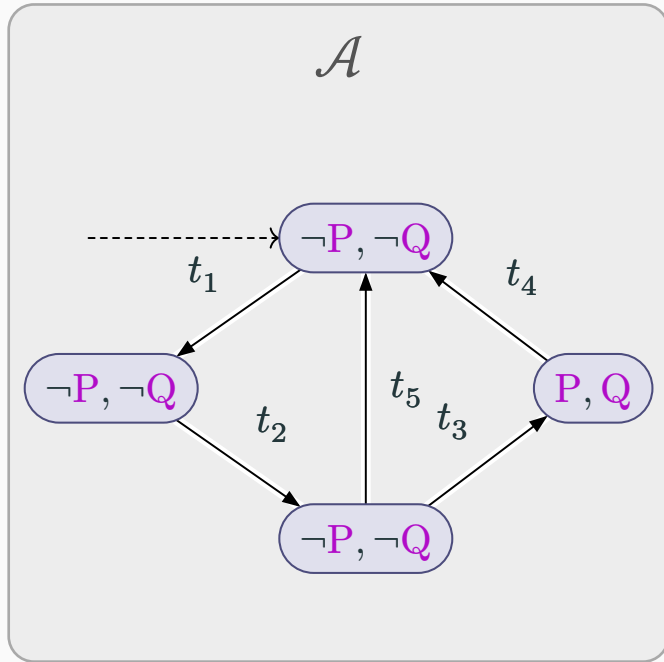


## Exercise: Constructing the automaton $\mathcal{B}_{\neg\varphi}$ (1)



Construct the automaton  $\mathcal{B}_{\neg\varphi}$  recognizing all executions **not satisfying**  $\varphi = \mathbf{G}(p \implies \mathbf{F}q)$ .

# Exercise: Constructing the automaton $\mathcal{B}_{\neg\varphi}$ (2)

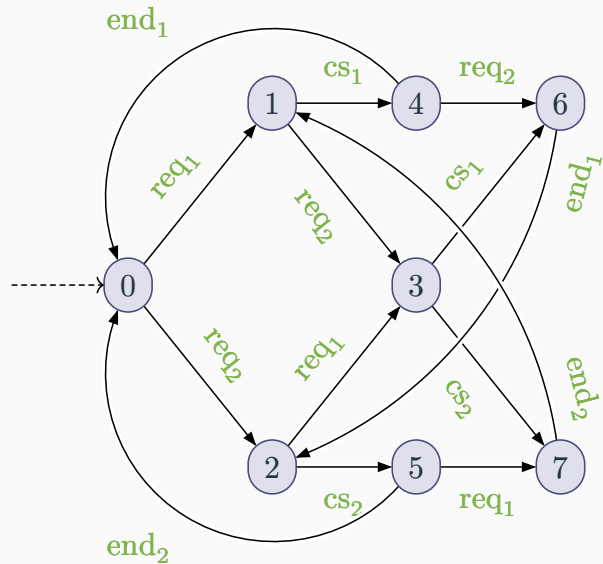


$$\varphi = \mathbf{G}(\mathbf{P} \implies \mathbf{XFQ})$$

Does  $\mathcal{A} \models \varphi$ ?



# Exercise: Mutual exclusion problem



$$\varphi = G \neg (CS_1 \wedge CS_2)$$

Does  $\mathcal{A} \models \varphi$ ?



Construct Büchi automata recognizing:

- ▶  $X\neg p \implies XXp$
- ▶  $pUG(q \vee r)$
- ▶  $(XXp)U(p \vee q)$

**It may be not necessary to construct the product automaton entirely**

- ▶ We build it step by step, and stop when we find a counter-example

LTL

Büchi automata

Constructing  $\mathcal{B}_{\neg\varphi}$

**Validating the formula**



Fairness

LTL

CTL

# Validating the formula

---

1. Construct the automaton  $\mathcal{B}_{\neg\varphi}$  recognizing all executions **not satisfying**  $\varphi$   
 $\mathcal{L}(\mathcal{B}_{\neg\varphi}) = [\neg\varphi]$  ✓
2. Test if  $\mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{B}_{\neg\varphi}) = \emptyset$ 
  - ▶ Construct the product automaton  $\mathcal{A} \times \mathcal{B}_{\neg\varphi}$  ✓
  - ▶ Check if  $\mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{B}_{\neg\varphi}) = \mathcal{L}(\mathcal{A} \times \mathcal{B}_{\neg\varphi})$  is empty
    - ▶ *if empty, then  $\mathcal{A} \models \varphi$*
    - ▶ Check if there exists a word accepted by  $\mathcal{A} \times \mathcal{B}_{\neg\varphi}$  ✓

LTL

Büchi automata

Constructing  $\mathcal{B}_{\neg\varphi}$

Validating the formula



**Fairness**

LTL

CTL

# Fairness



LTL

Büchi automata

Constructing  $\mathcal{B}_{\neg\varphi}$

Validating the formula



**Fairness**

LTL

CTL

LTL

---

## Fairness

A **fair execution** is an execution where every action that is enabled infinitely often is executed infinitely often

In the case of the mutual exclusion problem, we can express fairness as:

- ▶ Every process is enabled infinitely often:  $\bigwedge_i (\text{GF } \text{CS}_i)$
- ▶ No process remains infinitely long in the critical section:  $\bigwedge_i \neg(\text{FG } \text{CS}_i) = \bigwedge_i \text{GF}(\neg \text{CS}_i)$



## Fairness constraints

- ▶ **Unconditional** fairness constraint:  $\text{GF}\varphi$
- ▶ **Strong** fairness constraint:  $\text{GF}\varphi \implies \text{GF}\psi$
- ▶ **Weak** fairness constraint:  $\text{FG}\varphi \implies \text{GF}\psi$

A fairness **condition** is a conjunction of fairness **constraints**

- ▶ A fairness condition is an LTL formula!

Let  $\mathcal{A}$  be an automaton,  $fair$  a fairness condition and  $\varphi$  an LTL formula.

## Definition 1

We define a fair satisfaction relation  $\mathcal{A} \models_f \varphi$  as follows:

- ▶  $\mathcal{A} \models_f \varphi$  if for all *fair* runs  $\rho$  of  $\mathcal{A}$ ,  $\rho, 0 \models \varphi$ .
- ▶ equivalently,  $\mathcal{A} \models_f \varphi$  if for all runs  $\rho$  of  $\mathcal{A}$ ,  $\rho, 0 \models_f \varphi$ .

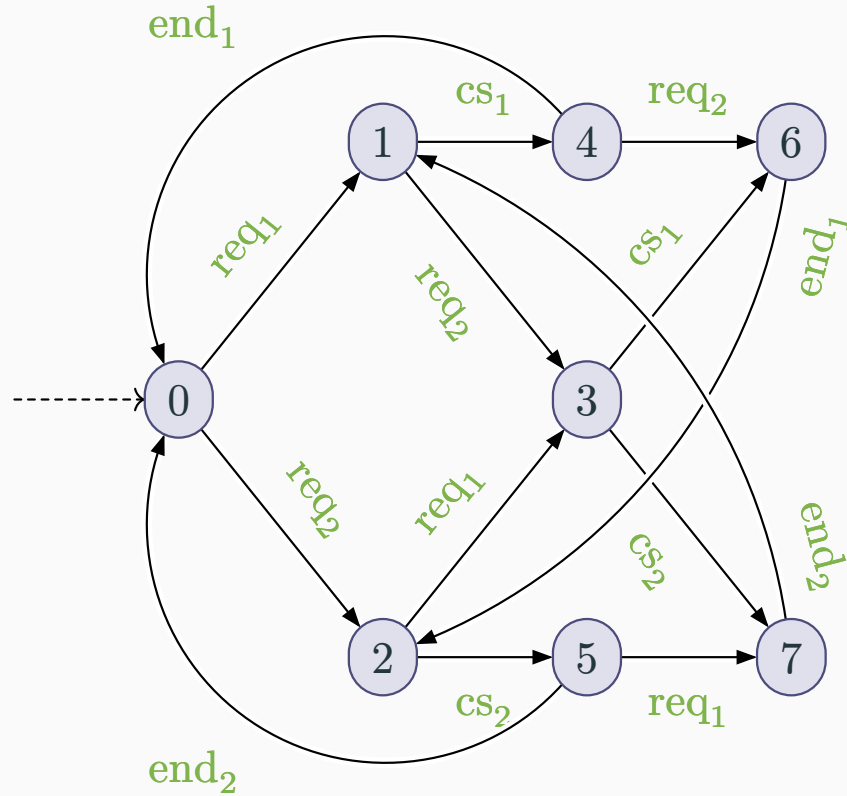


## Theorem 2 (LTL model checking)

$\mathcal{A} \models_f \varphi$  if and only if  $\mathcal{A} \models fair \implies \varphi$



# Example: Mutual Exclusion with fairness



$\mathcal{A} \not\models GFCS_1 \wedge GFCS_2$

but

$\mathcal{A} \models_f GFCS_1 \wedge GFCS_2$

LTL

Büchi automata

Constructing  $\mathcal{B}_{-\varphi}$

Validating the formula



**Fairness**

LTL

**CTL**

CTL

---

## Question



Can we express fairness in CTL?

## No, we can't!



- ▶ We would like to write  $A(\text{fair} \implies \varphi)$  or  $E(\text{fair} \wedge \varphi)$
- ▶ But they are not CTL formulas! (*They are CTL\* formulas*)

$\text{CTL}_f \ni \varphi ::= p \mid \neg \varphi \mid \varphi \vee \psi \mid E_f X \varphi \mid A_f X \varphi \mid E_f \varphi U \psi \mid A_f \varphi U \psi$

|                                     |                                                                                                                              |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| $\rho, i \models p$                 | if $p \in \text{lab}(\rho(i))$                                                                                               |
| $\rho, i \models \neg \varphi$      | if $\varphi, i \not\models \varphi$                                                                                          |
| $\rho, i \models \varphi \vee \psi$ | if $\rho, i \models \varphi$ or $\rho, i \models \psi$                                                                       |
| $\rho, i \models E_f \varphi$       | if there exists a <b>fair run</b> $\rho' : \rho(0) \dots \rho(i) = \rho'(0) \dots \rho'(i)$ s.t. $\rho', i \models \varphi$  |
| $\rho, i \models A_f \varphi$       | if for all <b>fair run</b> $\rho'$ s.t. $\rho(0) \dots \rho(i) = \rho'(0) \dots \rho'(i)$ we have $\rho', i \models \varphi$ |

## Question



How can we check a fair CTL property?

- ▶ We label the states with a *fair* property, which is true if a fair execution can reach the state.
- ▶ We check the formula  $\varphi$  against the model, using the following rules, depending on the fairness constraint.

- ▶ **Problem 1:** How to compute *fair*?
  - ▶  $\rho, i \models \text{fair}$  if there exists a **fair run** starting from  $i$ .
  - ▶ depending on the fairness constraint
- ▶ **Problem 2:** How to check  $E_f G \varphi$ ?

|                                      |                                                                                                                                                                                                                                       |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\rho, i \models E_f X \varphi$      | iff $\rho, i \models X \varphi$                                                                                                                                                                                                       |
| $\rho, i \models A_f X \varphi$      | iff $\rho, i \models AX(\neg \text{fair} \vee \varphi)$                                                                                                                                                                               |
| $\rho, i \models E_f \varphi U \psi$ | iff $\rho, i \models E \varphi U (\text{fair} \wedge \psi)$                                                                                                                                                                           |
| $\rho, i \models A_f \varphi U \psi$ | iff $\rho, i \models \neg E_f G \neg \psi \wedge \neg E_f (\neg \psi) U (\neg \varphi \wedge \neg \psi)$<br>iff $\rho, i \models \neg E_f G \neg \psi \wedge \neg E (\neg \psi) U (\text{fair} \wedge \neg \varphi \wedge \neg \psi)$ |

# Problem 1: Computing *fair*

## Definition 1 (Strongly Connected Components)

A **strongly connected component** (SCC) of a graph is a maximal subgraph such that for every pair of nodes  $(s, s')$ ,  $s'$  is reachable from  $s$  and  $s$  is reachable from  $s'$



Tarjan's algorithm can compute the SCCs of a graph in linear time



## Problem 1: Computing *fair*

**To compute *fair* for a fairness constraint  $fair = GF\varphi$**

- ▶ We mark the states with  $\varphi$
- ▶ We compute the SCCs of the model (Tarjan's algorithm)
- ▶ We compute the union  $S$  of the SCCs containing a state marked with  $\varphi$
- ▶ *fair* is true in the states from which we can reach  $S$

## Problem 2: Computing $E_f G \varphi$

- ▶ We mark the states with  $\varphi$
- ▶ We compute the SCCs of the model **restricted to the marked states**  $\mathcal{A}^\varphi$  (Tarjan's algorithm)
- ▶ We compute the union  $S$  of the SCCs containing a state marked with  $\varphi$
- ▶  $\mathcal{A}, q \models E_f G \varphi$  iff  $\mathcal{A}, q \models E \varphi U S$  iff  $\mathcal{A}^\varphi, q \models EFS$   
→ we reduce the problem to checking reachability in the subgraph  $\mathcal{A}^\varphi$

## Question



How can we check a fair CTL property?

- ▶ We label the states of the model with a new atomic property *fair*, which is true if a fair execution can start from this state
- ▶ We check the formula *fair*  $\implies \varphi$  using standard CTL model checking

|                                      |                                                                                                                                                                                                                                       |   |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| $\rho, i \models E_f X \varphi$      | iff $\rho, i \models EX \text{ fair} \wedge \varphi$                                                                                                                                                                                  | ✓ |
| $\rho, i \models A_f X \varphi$      | iff $\rho, i \models AX(\neg \text{ fair} \vee \varphi)$                                                                                                                                                                              | ✓ |
| $\rho, i \models E_f \varphi U \psi$ | iff $\rho, i \models E \text{ fair} U (\text{fair} \wedge \psi)$                                                                                                                                                                      | ✓ |
| $\rho, i \models A_f \varphi U \psi$ | iff $\rho, i \models \neg E_f G \neg \psi \wedge \neg E_f (\neg \psi) U (\neg \varphi \wedge \neg \psi)$<br>iff $\rho, i \models \neg E_f G \neg \psi \wedge \neg E (\neg \psi) U (\text{fair} \wedge \neg \varphi \wedge \neg \psi)$ | ✓ |

This presentation can be published, reused and modified under the terms of the license Creative Commons **Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**



[creativecommons.org/licenses/by-nc-sa/4.0/](https://creativecommons.org/licenses/by-nc-sa/4.0/)

Author: **Dylan Marinho**

*Inspired by the courses of Étienne André and  
Nathalie Sznajder*

